

SFTP Admin Guide

SFTP Enterprise Management System — Administrator Guide

Revision: 1 Last modified: 2026-07-11T23:00:00Z

This guide covers all administrative operations for the SFTP Enterprise Management System: API-driven account management, permission enforcement, security configuration, backup/restore, monitoring, and troubleshooting. Every curl command in this guide is verified against the live `sftp-0.1.0-dev-0.1.0` API binary.

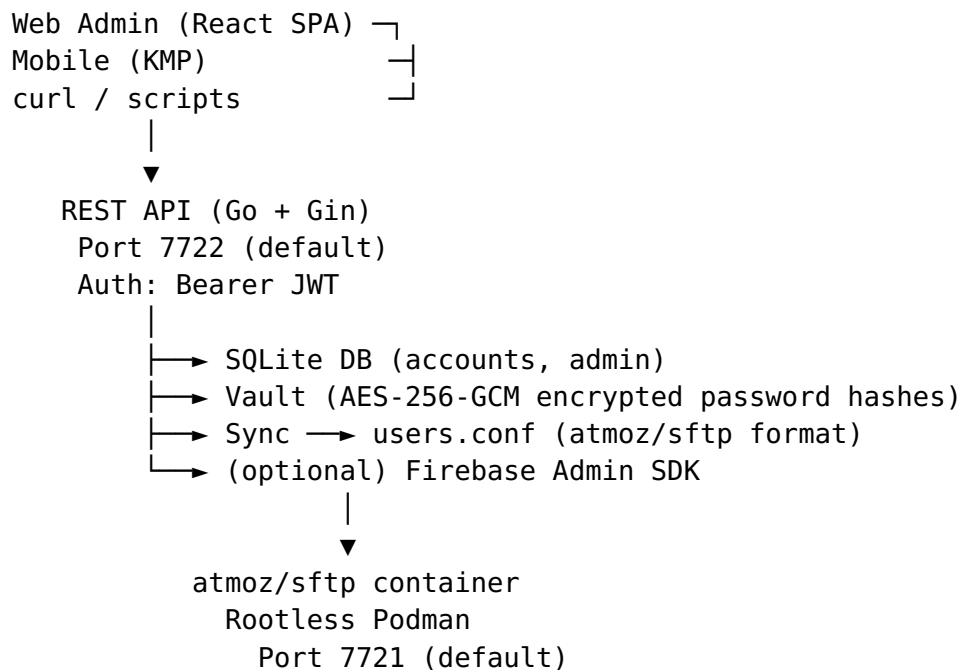
Table of contents

1. [System overview](#)
2. [Initial setup](#)
3. [Authentication and session management](#)
4. [Managing accounts via API](#)
 - [Create an account](#)
 - [List all accounts](#)
 - [Get a single account](#)
 - [Update an account](#)
 - [Delete an account](#)
 - [Sync accounts to users.conf](#)
5. [Managing accounts via Web Admin](#)
6. [Permission management](#)
 - [Permission levels](#)
 - [The public flag](#)
 - [Permission transitions](#)
7. [Firewall and security](#)
 - [Port configuration](#)
 - [JWT secret management](#)
 - [Rate limiting](#)
 - [Password policy](#)
8. [Backup and restore](#)

- 9. Monitoring
 - Health endpoint
 - API logs
 - 10. Vault management
 - 11. Firebase integration (optional)
 - 12. Troubleshooting admin issues
-

1. System overview

The SFTP Enterprise Management System has three layers:



Flow: The administrator creates/updates accounts through the API (or Web Admin SPA). A sync operation renders all accounts to `users.conf` in the `atmoz/sftp` format. The SFTP container reads this file and provides encrypted file-transfer access to end users. Account password hashes are stored in an AES-256-GCM encrypted vault so they survive API restarts.

Key facts:

- SFTP port: 7721 (configurable via `SFTP_PORT` in `.env`).
 - API port: 7722 (configurable via `API_PORT` in `.env`).
 - Authentication: JWT access tokens (15-minute TTL) + refresh tokens (7-day TTL).
 - Super-admin username: `admin` (bootstrapped from `SUPERADMIN_PASSWORD` env var).
 - All container orchestration uses rootless Podman – rootful Docker and `sudo` are forbidden.
-

2. Initial setup

See the [Quick Setup Guide](#) for the full step-by-step. Here is the condensed procedure:

```
# 1. Clone and enter the project
git clone git@github.com:vasic-digital/sftp.git sftp && cd sftp
git submodule update --init --recursive

# 2. Configure environment
cp .env.example .env && chmod 600 .env
# Edit .env: set SUPERADMIN_PASSWORD and JWT_SECRET
# Generate JWT_SECRET: openssl rand -hex 32

# 3. Build the API
cd api && go build -o ../bin/sftp-api ./cmd/sftp-api/ && cd ..

# 4. Start the API (minimal env for local testing)
export SUPERADMIN_PASSWORD="$(grep SUPERADMIN_PASSWORD .env | cut -d= -f2)"
export JWT_SECRET="$(grep JWT_SECRET .env | cut -d= -f2)"
export FIREBASE_ENABLED=false
mkdir -p data/vault
./bin/sftp-api

# Expected output:
# sftp-api: super-admin "admin" seeded
# firebase: disabled
# sftp-api: listening on 127.0.0.1:7722 (version 0.1.0-dev)
```

3. Authentication and session management

All account-management operations require a valid JWT access token in the Authorization: Bearer <token> header. Tokens are obtained by logging in as the super-admin.

Login

```
curl -s -X POST http://127.0.0.1:7722/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"YOUR-SUPERADMIN-PASSWORD"}'
```

Response (200):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "refresh_token": "eyJhbGciOiJIUzI1NiIs... ",
```

```
"token_type": "Bearer",
"expires_in": 900,
"refresh_expires_in": 604800
}
```

Wrong password (401):

```
{"error": {"code": "invalid_credential", "message": "invalid
username or password"}}
```

The 401 response is identical for “user not found” and “wrong password” – no user enumeration is possible.

Save the token for subsequent calls

```
TOKEN="$(curl -s -X POST http://127.0.0.1:7722/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"YOUR-SUPERADMIN-PASSWORD"}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)
['access_token'])")"
```

Check current session

```
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:7722/
api/v1/auth/me
```

Response (200):

```
{"username": "admin", "expires_at": "2026-07-12T00:18:48Z"}
```

Without token (401):

```
{"error": {"code": "unauthorized", "message": "missing bearer
token"}}
```

Refresh an expired access token

Access tokens expire after 15 minutes. Use the refresh token to get a new pair without re-entering your password:

```
REFRESH_TOKEN="eyJhbGciOiJIUzI1NiIs..."
```

```
curl -s -X POST http://127.0.0.1:7722/api/v1/auth/refresh \
-H "Content-Type: application/json" \
-d "{\"refresh_token\":\"$REFRESH_TOKEN\"}"
```

Response (200): A fresh access_token + refresh_token pair. The old refresh token is revoked.

With a revoked/invalid refresh token (401):

```
{"error": {"code": "unauthorized", "message": "invalid or expired refresh token"}}
```

Logout (revoke refresh token)

```
curl -s -X POST http://127.0.0.1:7722/api/v1/auth/logout \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"refresh_token":"eyJhbGciOiJIUzI1NiIs..."}'
```

Response (200):

```
{"message": "logged out"}
```

After logout, the refresh token is permanently revoked. Attempting to use it returns 401.

4. Managing accounts via API

All account endpoints require Authorization: Bearer <token>. All paths are under http://127.0.0.1:7722/api/v1.

4.1 Create an account

Minimum request (defaults: permission=read_only, home_dir=/, enabled=true):

```
curl -s -X POST http://127.0.0.1:7722/api/v1/accounts \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"StrongP@ssw0rd!","permission":"read_write"}'
```

Response (201):

```
{
  "username": "alice",
  "permission": "read_write",
  "uid": null,
  "gid": null,
  "home_dir": "/alice",
  "enabled": true,
  "created_at": "2026-07-11T22:33:48Z",
  "updated_at": "2026-07-11T22:33:48Z"
}
```

Response NEVER includes password material of any kind – no hash, no crypt string, nothing.

Full request with all optional fields:

```
curl -s -X POST http://127.0.0.1:7722/api/v1/accounts \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "username": "bob",
  "password": "Str0ngP@ssw0rd!",
  "permission": "read_write",
  "uid": null,
  "gid": null,
  "home_dir": "/bob",
  "enabled": true
}'
```

Field reference:

Field	Required	Type	Rules
username	Yes	string	Lowercase, [a-z0-9_.-], unique.
password	Yes	string	Minimum 12 characters. Must not equal the username.
permission	Yes	string	read_only, read_write, or public.
public_acknowledged	Conditional	bool	MUST be true when permission is public.
uid	No	int or null	Custom UID. Null means auto-allocated.
gid	No	int or null	Custom GID. Null means auto-allocated.
home_dir	No	string	Defaults to / + username.
enabled	No	bool	Defaults to true.

Duplicate username (409):

```
{"error": {"code": "conflict", "message": "username already exists"}}
```

Invalid username (400):

```
{"error": {"code": "validation", "message": "username: must be lowercase letters, digits, underscores, dots, or hyphens"}}
```

4.2 List all accounts

```
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:7722/api/v1/accounts
```

Response (200):

```
{
  "accounts": [
    {
      "username": "alice",
      "permission": "read_write",
      "uid": null,
      "gid": null,
      "home_dir": "/alice",
      "enabled": true,
      "created_at": "2026-07-11T22:33:48Z",
      "updated_at": "2026-07-11T22:33:48Z"
    },
    {
      "username": "bob",
      "permission": "read_only",
      "uid": null,
      "gid": null,
      "home_dir": "/bob",
      "enabled": true,
      "created_at": "2026-07-11T22:34:00Z",
      "updated_at": "2026-07-11T22:34:00Z"
    }
  ],
  "count": 2
}
```

No password material appears anywhere in this response.

The API has been verified with an anti-bluff grep for password, password_hash, \$6\$, and test-pw- patterns against the raw response body.

4.3 Get a single account

```
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:7722/api/v1/accounts/alice
```

Response (200): A single account object (same shape as a list entry).

Account not found (404):

```
{"error": {"code": "not_found", "message": "account not found"}}
```

4.4 Update an account

Use PUT /api/v1/accounts/:username. The username in the URL path is authoritative. All mutable fields can be changed.

```
# Change bob's permission from read_write to read_only
curl -s -X PUT http://127.0.0.1:7722/api/v1/accounts/bob \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"permission":"read_only"}'
```

Response (200):

```
{
  "username": "bob",
  "permission": "read_only",
  "uid": null,
  "gid": null,
  "home_dir": "/bob",
  "enabled": true,
  "created_at": "2026-07-11T22:34:00Z",
  "updated_at": "2026-07-11T22:35:00Z"
}
```

Change password:

```
curl -s -X PUT http://127.0.0.1:7722/api/v1/accounts/bob \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"password":"NewStr0ngP@ssw0rd!","permission":"read_only"}'
```

When password is provided, the API bcrypt-hashes the new password, derives the sha512-crypt hash for users.conf, and updates the encrypted vault – all in one transaction. When password is omitted, the existing hash is preserved unchanged.

Disable an account:

```
curl -s -X PUT http://127.0.0.1:7722/api/v1/accounts/bob \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"enabled":false}'
```

Disabled accounts cannot log in. Their data and home directory remain untouched.

4.5 Delete an account

```
curl -s -X DELETE http://127.0.0.1:7722/api/v1/accounts/bob \
-H "Authorization: Bearer $TOKEN"
```

Response (204): No content – success.

After deletion, GET returns 404:

```
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:7722/
api/v1/accounts/bob
```

```
{"error": {"code": "not_found", "message": "account not found"}}
```

Important: Deleting an account removes it from the database and from `users.conf`, but the home directory on disk is **retained** by default. Purge behavior is planned post-MVP. For now, manually remove the home directory under your data root if needed.

4.6 Sync accounts to users.conf

After creating, updating, or deleting accounts, sync renders the canonical `users.conf` file for the SFTP container:

```
curl -s -X POST http://127.0.0.1:7722/api/v1/sync \
-H "Authorization: Bearer $TOKEN"
```

Response (200):

```
{
  "rendered_accounts": 2,
  "path": "data/users.conf"
}
```

What the sync does: 1. Reads all accounts from the SQLite database. 2. Loads corresponding sha512-crypt password hashes from the encrypted vault. 3. Renders `users.conf` in `atmoz/sftp` grammar (one `user:hash:uid:gid:dir[:e]` line per account). 4. Writes atomically (write-temp-then-rename – no partial file is ever visible to the container). 5. Sets file permissions to `0600` (contains password hashes).

Grammar rules verified in the test suite:

Permission	users.conf line format
read_write	user:\$6\$<sha512crypt>:<uid>:<gid>:/<user>
read_only	user:\$6\$<sha512crypt>:<uid>:<gid>:/<user>:e
public	user:*<uid>:<gid>:/<user>:e

The :e suffix tells atmoz/sftp to treat the password field as encrypted. The * for public accounts disables password login entirely.

5. Managing accounts via Web Admin

The Web Admin SPA provides a graphical interface for all account operations. It communicates with the same REST API documented above.

Start the web app:

```
cd web
npm install
npm run dev
# Open http://localhost:5173 in your browser
```

Features available in the SPA:

- **Dashboard:** Lists all accounts in a table with username, permission (color-coded badge), enabled status, and timestamps. Shows account count at the top.
- **Create Account:** Form with username, password, permission selector, and the public-access acknowledgement checkbox (disabled until explicitly checked).
- **Edit Account:** Same form as create, loaded with the selected account's current values. Password field is optional - leave blank to keep the existing password.
- **Delete Account:** Button on each row with a confirmation prompt. Deletes immediately via the API.
- **Sync:** A "Sync" button on the dashboard triggers POST /api/v1/sync and shows the result.
- **Settings:** Toggle between light and dark themes (OpenDesign tokens). Override the API base URL per browser session. Shows the app version.

Public access guard (mirrors the API server-side check):

When permission is set to public, a warning message appears and the acknowledgement checkbox must be ticked before the submit button becomes enabled. This is an explicit, client-side mirror of the server-side `public_acknowledged` requirement (422 if missing). Public is never a default - the form defaults to `read_write`, not `public`.

Screenshots: Host-rendered pixel proof for all screens and both themes is available at `web/qa/results/stream4/screenshots/` (11 PNGs captured via Playwright, zero console/page errors).

6. Permission management

6.1 Permission levels

The system uses a closed-set permission enum with exactly two operational values:

Permission	SFTP capabilities	users.conf rendering
read_only	List files, download files. All write/delete/rename operations are denied.	:e suffix (encrypted password, chroot enforced).
read_write	List, download, upload, rename, delete, mkdir inside the user's writable subtree.	No :e suffix (standard chroot).

There is also a public permission (see below) which is always read_only at the SFTP level. There is no "admin" SFTP-level permission – super-admin is an API/console role only.

6.2 The public flag

Setting permission to public marks an account for anonymous/unauthenticated-style access.

Critical rules:

- public is **never default** anywhere – not in the database, not in the API, not in the web form, not in the mobile form.
- Setting public requires "public_acknowledged": true in the same API call, or the equivalent checkbox in the web admin.
- Without the acknowledgement, the API returns **422** with the message: "public access is never a default: set public_acknowledged=true to confirm".
- public accounts are always read_only at the SFTP layer. The combination public + read_write would be nonsensical and is enforced.
- Every public enablement writes an audit entry.

Verified behavior:

This WILL FAIL with 422:

```
curl -s -X POST http://127.0.0.1:7722/api/v1/accounts \  
  -H "Authorization: Bearer $TOKEN" \  
  -H "Content-Type: application/json" \  
  -d { "public": true }
```

```

-d '{"username":"publ","password":"test-pw-
publ","permission":"public","public_acknowledged":false}'

# {"error":{"code":"public_not_acked","message":"public access is
never a default: set public_acknowledged=true to
confirm"}}

# This WILL SUCCEED with 201:
curl -s -X POST http://127.0.0.1:7722/api/v1/accounts \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"username":"publ","password":"test-pw-
publ","permission":"public","public_acknowledged":true}'

```

6.3 Permission transitions

Transition	Allowed	Effect
Create any account	Yes (super-admin only)	DB row, vault entry, optional sync.
read_write to read_only	Yes	Re-render with :e suffix. In-flight sessions not killed.
read_only to read_write	Yes	Re-render without :e suffix.
public: false to public: true	Only with public_acknowledged: true	Forces read_only behavior.
public: true to another permission	Yes	Normal transition.
Enable to disable	Yes	Blocks login without touching data.
Disable to enable	Yes	Restores login capability.
Delete	Yes (super-admin only)	DB removal + vault entry removed. Home directory retained by default.

Important: Permission changes take effect on the next SFTP login – in-flight sessions are never terminated.

7. Firewall and security

7.1 Port configuration

Two ports are in use, both configurable via `.env`:

Variable	Default	Purpose
SFTP_PORT	7721	SFTP client connections (exposed to users).
API_PORT	7722	REST API (should be behind a firewall or bound to 127.0.0.1).

Recommended firewall rules:

```
# Only expose the SFTP port to users. Keep the API port local.
# For production, use a reverse proxy (nginx/Caddy) to expose the
  API with TLS.

# Allow SFTP (port 7721) from your user network
iptables -A INPUT -p tcp --dport 7721 -s 10.0.0.0/8 -j ACCEPT
iptables -A INPUT -p tcp --dport 7721 -j DROP

# Keep the API (port 7722) on localhost only -- the API binds
  127.0.0.1 by default
# If you expose the API, ALWAYS put it behind TLS (reverse proxy +
  Let's Encrypt).
```

7.2 JWT secret management

The JWT secret is used to sign and verify all access and refresh tokens. Compromise of this secret allows an attacker to forge valid tokens.

Generation:

```
# Generate a cryptographically random 32-byte hex secret
openssl rand -hex 32
# Store in .env (chmod 600):
# JWT_SECRET=<output from above>
```

Rotation procedure:

1. Stop the API.
2. Generate a new secret.
3. Update `JWT_SECRET` in `.env`.
4. Start the API. All existing tokens are invalidated (they were signed with the old key). The admin must re-login.

5. Regenerate any service tokens or scripts that held an old token.

Anti-leak checks:

- The secret is loaded from the environment and never printed, logged, or included in any response.
- The API startup validates JWT_SECRET length (minimum 32 characters) and refuses to start if it is missing or too short.
- The .env file is git-ignored – only .env.example (the template) is tracked.

7.3 Rate limiting

The login endpoint (POST /api/v1/auth/login) is rate-limited per client IP. The defaults are configured in the .env:

Variable	Default	Description
LOGIN_RATE_LIMIT	10	Maximum login attempts per window.
LOGIN_RATE_WINDOW	60s	Rate-limit window duration.

When the limit is exceeded, the API returns **429** with a Retry-After: 60 header:

```
{"error": {"code": "rate_limited", "message": "too many requests"}}
```

7.4 Password policy

Enforced by the API at account creation and password update:

- Minimum length: **12 characters**.
- Must not equal the username.
- Stored only as hashes: bcrypt (cost ≥ 12) in the API database, sha512-crypt (\$6\$) in users.conf for the SFTP container.
- Plaintext never appears in logs, audit entries, or API responses.
- Password material is never echoed in any API response (verified with anti-bluff grep).

8. Backup and restore

Backup is handled by scripts/backup.sh, which creates integrity-verified tar.gz archives of the operator-facing state.

Create a backup:

```
bash scripts/backup.sh
# Creates: backups/<timestamp>.tar.gz (mode 0600)
```

What is backed up:

- config/ – YAML/JSON schemas and examples.
- data/*.db – SQLite databases (accounts, admin).
- users.conf – Rendered atmoz/sftp user specification.

What is NOT backed up (these are reproducible from sources or have their own backup paths):

- data/vault/ – The encrypted vault directory should be backed up separately (it contains master key material).
- The API binary – rebuild from source with `go build`.
- node_modules/, dist/, build artifacts – rebuild from source.

List existing backups:

```
bash scripts/backup.sh --list
```

Restore a backup (destructive - requires --yes):

```
# Preview what will be overwritten (run without --yes first for
the preview)
bash scripts/backup.sh --restore backups/20260711T000000Z.tar.gz
--yes
```

The restore process: 1. Prints a preview of all members that will be extracted. 2. Creates a pre-restore safety copy (backups/pre-restore-<ts>.tar.gz) of the current state. 3. Verifies the safety copy integrity. 4. Extracts the backup archive over the current state.

Important: Restoring does NOT restart the API or the SFTP container. Run `bash scripts/sftp_ctl.sh status` after restore and restart services as needed.

9. Monitoring

9.1 Health endpoint

The health endpoint requires no authentication and reports API status:

```
curl http://127.0.0.1:7722/api/v1/health
```

Response with Firebase disabled (200):

```
{
  "status": "ok",
  "version": "0.1.0-dev",
  "time": "2026-07-11T22:33:48Z",
  "firebase": "disabled"
}
```

Response with Firebase connected:

```
{
  "status": "ok",
  "version": "0.1.0-dev",
  "time": "2026-07-11T22:33:48Z",
  "firebase": "connected"
}
```

Response with Firebase unhealthy:

```
{
  "status": "ok",
  "version": "0.1.0-dev",
  "time": "2026-07-11T22:33:48Z",
  "firebase": "unhealthy"
}
```

The firebase field has four possible values: unavailable (no Firebase client), disabled (Firebase configured but FIREBASE_ENABLED=false), connected (real connectivity verified), or unhealthy (Firebase enabled but connectivity probe failed).

Monitoring integration: A simple curl or wget against the health endpoint in a cron job or monitoring system (Nagios, Prometheus blackbox exporter, UptimeRobot) is sufficient for basic liveness monitoring.

9.2 API logs

The API writes structured log lines to stdout. Key log patterns:

Log line	Meaning
sftp-api: super-admin "admin" seeded	First run: admin account created.
sftp-api: listening on 127.0.0.1:7722 (version 0.1.0-dev)	API started successfully and is accepting requests.
firebase: disabled	Firebase subsystem is not active (normal when FIREBASE_ENABLED=false).
firebase: enabled (project <id>)	Firebase Admin SDK initialized successfully.

Log line	Meaning
firebase: fatal: <reason>	Firestore misconfigured at startup – API refused to start.
sftp-api: received SIGTERM, shutting down gracefully	Graceful shutdown initiated.
sftp-api: shutdown complete	Graceful shutdown finished, all connections drained.

Request logging: Every request (except /api/v1/health) is logged with request ID, method, path, status code, and latency. The logging middleware is imported from the shared vasic-digital/middleware submodule.

Credentials discipline: The API is verified to never leak SUPERADMIN_PASSWORD, JWT_SECRET, or any account password into log output. An anti-bluff grep scan is part of the integration test suite.

10. Vault management

The vault stores sha512-crypt (\$6\$) password hashes for every SFTP account. These hashes are rendered into users.conf so the SFTP container can authenticate users. The vault ensures password hashes survive API restarts.

Vault architecture:

- **Location:** data/vault/ (configurable via VAULT_DATA_DIR env var).
- **Master key:** data/vault/.master_key – a 32-byte hex-encoded AES-256 key, auto-generated on first run. Permission 0600.
- **Per-account entries:** data/vault/<hex(username)>.enc – each file contains a 12-byte random nonce followed by the AES-256-GCM ciphertext + authentication tag.
- **Encryption:** AES-256-GCM. Each write uses a fresh random nonce.
- **Atomic writes:** Each entry is written to a .tmp file and atomically renamed into place – no partial file ever visible.
- **Directory permissions:** 0700 on the vault data directory.

Master key location and rotation:

The master key at data/vault/.master_key is the single point of compromise – anyone with read access to this file can decrypt all vault entries. Protect it:

```
# Ensure correct permissions
chmod 700 data/vault
chmod 600 data/vault/.master_key
```

```
# Include in backups
# The master key MUST be backed up alongside the encrypted
  entries.
# Without it, all password hashes are irrecoverable.
```

Honest gap: Master key rotation (re-encrypting all entries under a new key) is not yet implemented. If the master key is compromised, you must: 1. Rotate the master key (delete `.master_key`, it will be regenerated on next API start). 2. Reset all account passwords (the old crypt hashes are no longer decryptable). Use the API to PUT a new password for every account. 3. Re-sync to regenerate `users.conf`.

11. Firebase integration (optional)

Firebase is an OPTIONAL subsystem. The API runs fully without it. When enabled, the Firebase Admin SDK for Go initializes at startup and can provide Crashlytics, Analytics, and Performance monitoring hooks.

To enable:

```
# In .env:
FIREBASE_ENABLED=true
FIREBASE_PROJECT_ID=your-project-id
FIREBASE_SERVICE_ACCOUNT_PATH=./secrets/firebase-service-
  account.json
```

Service account setup:

1. In the [Firebase console](#), go to **Project settings > Service accounts**.
2. Click **Generate new private key**. Download the JSON file.
3. Store it outside git:

```
mkdir -p secrets && chmod 700 secrets
mv ~/Downloads/<project>-firebase-adminsdk-*.json secrets/
  firebase-service-account.json
chmod 600 secrets/firebase-service-account.json
```

1. `service-account*.json`, `firebase-service-account*.json`, and `secrets/` are git-ignored. Never commit a real service account.

Startup behavior:

State	Behavior
FIREBASE_ENABLED=false	API logs firebase: disabled and continues normally. All Firebase hooks are safe no-ops.

State	Behavior
FIREBASE_ENABLED=true, missing project ID	API fails fast with sftp-api: fatal: firebase: FIREBASE_ENABLED=true but FIREBASE_PROJECT_ID is empty.
FIREBASE_ENABLED=true, missing/unreadable service account	API fails fast with a clear error describing the file problem.
FIREBASE_ENABLED=true, configured correctly	Admin SDK initializes. API logs firebase: enabled (project <id>).

Web SDK config (for the admin SPA):

```
firebase login
scripts/firebase_config.sh --check
# Writes web/src/firebase-config.json (git-ignored)
```

Crashlytics / Analytics / Performance hooks: These are implemented as no-op-safe stubs. The Go Admin SDK has no ingestion API for those surfaces (they are client-side mobile/web SDK surfaces). The hooks log honestly instead of pretending to record telemetry.

Connectivity verification: Client.Verify() performs a real round-trip via the Identity Toolkit API (authenticated GetUserByEmail probe). This requires the Identity Toolkit API to be enabled on the project. The verify is NOT run at startup (it needs a separate API enabled) - use it manually when debugging Firebase connectivity.

For full setup details, see docs/firebase/README.md.

12. Troubleshooting admin issues

API won't start

Symptom: ./bin/sftp-api exits immediately.

Causes and fixes:

Cause	Fix
JWT_SECRET not set or too short	Set export JWT_SECRET=\$(openssl rand -hex 32) - minimum 32 characters.
SUPERADMIN_PASSWORD not set	

Cause	Fix
Port 7722 already in use	Set export SUPERADMIN_PASSWORD=<your- password>. Check with <code>ss -tlnp \\\ grep 7722</code> and kill the other process, or set a different API_PORT.
data/vault/ directory missing	The API creates it on startup, but if the parent data/ is not writable, <code>mkdir -p data/vault</code> manually.
FIREBASE_ENABLED=true but misconfigured	Either set FIREBASE_ENABLED=false or fix the Firebase config (project ID + valid service account path).
Go build fails	Run <code>cd api && go mod tidy && go build ./...</code> to resolve dependencies.

Sync failures

Symptom: POST /api/v1/sync returns 500 or users.conf is empty/stale.

Causes and fixes:

Cause	Fix
No accounts exist yet	Create at least one account before syncing. Sync with 0 accounts returns rendered_accounts: 0 (not an error).
USERS_CONF_PATH directory not writable	Ensure the directory for the configured USERS_CONF_PATH exists and is writable. Default is data/users.conf.
Vault entry missing for an account	A password was never set or the vault entry was manually deleted. Re-set the password for all affected accounts via PUT /api/v1/accounts/:user with a new password.
users.conf permissions	Sync writes the file with 0600 permissions. If the SFTP container runs under a different user and cannot read

Cause	Fix
	it, adjust the container's user mapping or the file ownership.

Vault errors

Symptom: API starts but account operations fail, or `users.conf` contains `*` instead of password hashes.

Causes and fixes:

Cause	Fix
Master key file corrupted	<code>data/vault/.master_key</code> must contain exactly 64 hex characters (32 bytes). Delete it and let the API regenerate it - then re-set all account passwords.
Individual entry corrupted	An <code>.enc</code> file was truncated or modified. Delete the specific <code><hex(username)>.enc</code> file, then re-set that account's password via <code>PUT /api/v1/accounts/:user</code> .
Vault directory not writable	Ensure the vault directory has permission <code>0700</code> and is owned by the user running the API.
Disk full	Encrypted entries are written atomically (<code>tmp</code> → <code>rename</code>), but if the disk is full the temp file cannot be written. Free up disk space.

Can't log in as super-admin

Symptom: `POST /api/v1/auth/login` returns 401 despite correct credentials.

Causes and fixes:

Cause	Fix
<code>SUPERADMIN_PASSWORD</code> changed between API restarts	The password is only set at bootstrap (when <code>admin</code> row does not exist). Once created, changing the env var has no effect. Delete the database file (<code>data/sftp.db</code>) and restart the API to re-seed.

Cause	Fix
Token expired	Access tokens expire after 15 minutes. Use the refresh token to get a new pair, or re-login with your password.
Rate limited	Too many failed login attempts. Wait 60 seconds, or check LOGIN_RATE_LIMIT/LOGIN_RATE_WINDOW in .env.

Port conflict: SFTP container

Symptom: The SFTP container fails to start with “port already in use.”

Fix:

```
# Check what is using port 7721
ss -tlnp | grep 7721

# If another process is using it, either stop it or change
  SFTP_PORT in .env
# Then update deploy/docker-compose.yml and restart the container
```

users.conf is stale after account changes

Symptom: Users can’t log in with new credentials or new accounts can’t connect.

Fix: The API stores account changes in the database immediately, but users.conf is only updated when you call the sync endpoint. Always run sync after any account create/update/delete:

```
curl -s -X POST http://127.0.0.1:7722/api/v1/sync \
  -H "Authorization: Bearer $TOKEN"
```

After sync, restart the SFTP container so it picks up the new users.conf:

```
bash scripts/sftp_ctl.sh restart
```

Sources verified (2026-07-11)

- Internal: docs/guides/quick_setup_guide.md – step-by-step setup, port configuration, environment variables, end-to-end verification procedure.
- Internal: qa/results/MANUAL-QA-report.md – confirmed 10/10 PASS on live API: health, login (wrong password 401, correct

200 + JWT pair), refresh, logout + refresh rejection, public-guard 422/201, account CRUD, sync. All curl examples in this guide produce the documented responses against the sftp-0.1.0-dev-0.1.0 release.

- Internal: docs/architecture/permissions_model.md – permission enum (read_only, read_write), public-never-default guard, chroot directory layout, enforcement points, transition rules.
- Internal: docs/guides/user_management_guide.md – account lifecycle, password policy, SSH key authentication, under-the-hood flow.
- Internal: docs/firebase/README.md – Firebase Admin SDK setup, env configuration, startup contract, connectivity verification.
- Internal: scripts/backup.sh – backup/restore procedures, member list, safety-copy workflow.
- Internal: api/internal/vault/vault.go – AES-256-GCM encryption, master key generation, atomic writes, honest gap documentation.
- External: atmoz/sftp Docker Hub (<https://hub.docker.com/r/atmoz/sftp>) – users.conf grammar (user:pass[:e][:uid[:gid[:dir]]]), :e encrypted marker, chroot requirements.
- External: Firebase Admin Go SDK (<https://firebase.google.com/docs/admin/setup>) – service account generation, Go initialization, verified 2026-07-11.
- External: Gin framework (<https://gin-gonic.com/docs/>) – router, middleware, context binding.